Сређивање података

Прављење излазне варијабле:

Проберимо да ли има недостајућих вредности у варијабли од које треба да направимо излазну (нпр. правимо highprice на основу price, онда price не сме да има недостајућих вредности јер нећемо моћи лепо да направимо излазну). Ако има недостајућих вредности у тој варијабли онда уклањамо те редове, јер ако их допунимо (нпр. доминантном класом за факторске или медијаном за нумеричке) то не приказује стварну вредност наше варијабле, самим тим ни излазна неће бити репрезентативна. Брисање редова се ради на следећи начин:

```
dataSub<-dataSub[complete.cases(dataSub[,13]),]
```

НАПОМЕНА: data$naziv<-NULL , се ради само за колоне, за редове мора ово изнад

Кад смо сигурни да нема недостајућих можемо правити излазну варијаблу преко ifelse, унутар заграда стављамо одговарајући услов (дат у задатку). Кад се направи излазна варијабла, брише се варијабла од које је настала зато што она директно утиче на излазну и нећемо имати реалне предикције, него ћемо 100% тачно предвиђати, а то не води реалним резултатима.

На крају излазну варијаблу пребацујемо у факторску. Кораци креирања нове варијабле дати су на слици:

```
#1. zahtev kreiranje nove varijable
table(data$Exited)          #ima samo vrednosti 0 i 1
sum(is.na(data$Exited))     #nema nedostajucih
#mogu da napravim varijablu
data$Stayed<-ifelse(data$Exited==0,"yes","no")
#pretvaram odmah u faktorsku i brisem exited jer nema smisla da postoji vise
data$Stayed<-as.factor(data$Stayed)
data$Exited<-NULL
```

Избор варијабли које сматрамо значајним за предикцију:

Варијабле које су нпр ид, редни број, наслов, назив могу одмах да се избаце јер нема смисла да предикција зависи од наслова рецимо књиге, свака има разлициту вредност и глупо је да вршимо било какве предикције, јер нема утицаја на излазну променљиву.

За варијабле које нису очигледне проверавамо колико различитих вредности имају преко следеће наредбе, користимо apply кад желимо да неку фју применимо на све варијабле. Други параметар је 1 ако желимо да посматрамо редове, 2 ако желимо да посматрамо колоне.

```
apply(dataSub, 2, function(x) length(unique(x)))
```

У суштини варијабле које имају преко 10% различитих вредности (у односу на цео датасет, рецимо имамо 3000 опсервација, до 300 различитих вредности ћемо прихватати) могу се избацити. Избацивање варијабле се врши на следећи начин:

```
dataSub$Point.Earned<-NULL
```

Смисао постојања варијабли у датасету може се проверити (и пожељно је),преко графикона. За **факторске** варијабле график цртамо на следећи начин:

```
ggplot(dataSub, aes(x=Geography,fill = Stayed))+geom_bar()
```

За **нумеричке** користимо следећи график:

```
ggplot(dataSub,aes(x=Price,fill = High_reviews))+geom_density(alpha=0.5)
```

Параметар alpha означава провидност. На основу овако креираних графикона можемо видети да ли постоје или не постоје разлике у погледу излазне варијабле у зависности од различитих вредности за улазну варијаблу.

Провера и допуна недостајућих вредности:

Користимо следеће наредбе:

```
apply(dataSub,2,function(x) sum(is.na(x)))
apply(dataSub,2,function(x) sum(x=="",na.rm = T))
apply(dataSub,2,function(x) sum(x==" ",na.rm = T))
apply(dataSub,2,function(x) sum(x=="-",na.rm = T))
```

Ако имамо превише недостајућих вредности ту варијаблу одмах избацимо, јер допуном вредности за велики део опсервација правимо илузију јер ће подаци бити „идеални“ (нпр. медијана или доминантна класа).

Уколико имамо вредности типа „“, „ “, “-“, њих прво пребацујемо у NA јер је то једина недостајућа коју језик препознаје као недостајућу вредност. А затим допуњавамо.

```
data$Price[data$Price=="-"]<-NA
```

Допуњавање за **нумеричке** вредности:

```
data$Price<-as.numeric(data$Price) #mora prvo u numeric da bi moga shapiro da se radi
#shapiro.test(data$Price) #baca gresku jer je maksimum 5000 opservacija
shapiro.test(head(data$Price,n=5000)) # p vrednost je manja od 0.05 odbacujem nultu hipotezu, nema normalne raspodele
#zamena se vrsi medijanom jer nije norm raspodela
data$Price[is.na(data$Price)]<-median(data$Price,na.rm = T) # kod medijane ignorisem na vrednosti
```

У случају да је нормална расподела, мења се средњом вредношћу.

Допуњавање за **факторске** се врши преко доминантне класе:

```
dataSub$Geography[dataSub$Geography=="-"]<-NA
table(dataSub$Geography) #dominantna vrednost je france pa cu ove nedostajuce vrednosti njome zameniti
dataSub$Geography[is.na(dataSub$Geography)]<-"France"
```

Е уколико имамо **КАРАКТЕР** варијаблу њу можемо пребацити у факторску ако има мало вредности и категоричка је као нпр. пол, ако имамо неке бројеве који су само представљени преко карактер варијабле и за које има смисла радити рецимо медијану, средњу вредност, минимум, максимум... њих можемо пребацити у нумеричке.

НАПОМЕНА: 0 поена на испиту је ако типичну категоричку варијаблу пребацимо у нумеричку и даље се са њом понашамо као да је нумеричка.

Још неке форице што су увалили на роковима за сређивање података:

ОВО ЋУ ДОПУНИТИ НАКНАДНО КАД ПРОЂЕМ ЈОШ РОКОВА

Стабла одлучивања

Тип података: улазне варијабле – факторске или нумеричке

излазна варијабла- факторска

Алгоритам:

1. прављење тренинг и тест сета (createDataPartition u cheatsheet-u)
2. прављење модела (rpart u cheatsheet-u)
3. крос валидација (cross-validation u cheatsheet-u)
4. прављење предикције (predictions u cheatsheet-u)
5. прављење конфузионе матрице (САМОСТАЛНО ПРАВИМО)
6. евалуација метрика (САМОСТАЛНО ПРАВИМО)
7. креирање новог модела са најбољим CP параметром

У наставку је дат пример за сваки од корака:

```
library(caret)
set.seed(1010)
indexes <- createDataPartition(dataSub$Stayed, p = 0.8, list = FALSE)
treningSet <- dataSub[indexes, ]
testSet<- dataSub[-indexes,]
```

```
#5. zahtev cross validacija
library(e1071)
library(caret)
library(rpart)
folds= trainControl(method = "cv", number = 10)
cpGrid= expand.grid(.cp =seq(from = 0.01, to = 0.05, by = 0.001))
tree1.cv<-train(
      x = treningSet[,-8],
      y = treningSet$Stayed,
      method = "rpart",
      control = rpart.control(minsplit =20),
      trControl = folds,
      tuneGrid = cpGrid)

tree1.cv$bestTune$cp #najbolja vrednost cp parametra je 0.01

#6.zahtev kreiranje modela
library(rpart.plot)
tree1 <- rpart(Stayed ~.,data = treningSet,
                method = "class",                                    # build classification tree
                control = rpart.control(cp =tree1.cv$bestTune$cp))
rpart.plot(tree1,extra=104) # na osnovu grafika moze se videti tacno kako se drvo grana i kako donosi zakljuck
```

```
#da bih napravila matricu konfuzije treba mi predikcija za tree1
tree1.pred<- predict(object =tree1,
                     newdata =testSet,
                     type = "class")
#kreiranje matrice konfuzije
tree1.matrica<-table(true=testSet$Stayed,predicted=tree1.pred)
tree1.matrica
```

НАПОМЕНА: Код фје за евалуационе метрике прва 4 параметра се мењају у зависности од тога шта је у задатку дато као позитина класа!!!

```
evaluateMetrics<- function(cm){
  TP<-cm[2,2]
  TN<-cm[1,1]
  FP<-cm[1,2]
  FN<-cm[2,1]

  a<-sum(diag(cm))/sum(cm)
  p<-TP/(TP+FP)
  r<-TP/(TP+FN)
  F1<-(2*r*p)/(r+p)
  c(accuracy=a, precision=p,recall=r,F1=F1)

}

evaluateMetrics(tree1.matrica)
```

ТЕОРИЈА КОЈА ЈЕ БИТНА ЗА КОМЕНТАРИСАЊЕ РАДА:

из евалуациона матрице гледамо 4 вредности TP, TN, FP и FN:
True-Positive (TP) – колико клијената смо предвидели да су остали у банци и они су заиста остали, у нашем случају 749
True-Negative (TN) – колико клијената смо предвидели да су напустили банку и они су је заиста напустили, у нашем случају 91
False-Positive (FP) – колико клијената смо предвидели да су остали у банци а они су је ипак напустили, у нашем случају 113
False-Negative (FN) – колико клијената смо предвидели да су напустили банку, а они су ипак остали, у нашем случају 25
рачунамо 4 евалуационе метрике за наш модел:
accuracy – колико тачно предвиђамо излазну варијаблу, у нашем случају од 978 клијената ми смо тачно предвидели 807,односно, за 85.9% клијената смо тачно предвидели да ли ће остати у банци или не
precision – од свих клијената које смо предвидели да ће остати у банци, који проценат клијената је заиста остао, како нам је FP 126, такође видимо да нам је ова метрика висока и износи 0.87; односно, од свих клијената за које смо предвидели да ће остати у банци, 87% њих је стварно остало
recall – од свих клијената који су заиста остали у банци, који проценат клијената смо ми предвидели да ће остати у банци,како нам је FN само 45, можемо видети да нам је и ова метрика висока односно 0.968; односно, међу свим клијентима који су заиста остали у банци, ми смо тачно предвидели 96.8% њих
F1 – представља меру перформански модела за балансиране вредности precision и recall, и за њом постоји потреба јер су precision и recall у антагонистичком односу, повећање једне, друга се смањује и обрнуто
како су и precision и recall високи, и вредност ове метрике је високих 0.92. У контексту задатка, ова метрика показује да смо предвиђањима не само у највећој мери обухватили све клијенте који су остали у банци, већ и да нисмо правили много грешака предвиђајући останак у банци оних који су је заправо напустили.

ВАЖНО: дате интерпретације тј. начин на који су исказане није једини коректан начин, већ показује само један од начина да се формулишу интерпретације метрика како би се одговорило на захтев задатка.

К најближих суседа

Тип података: улазне варијабле – нумеричке (ако немају норм расп СТАНДАРДИЗАЦИЈА)

излазна варијабла- факторска

Алгоритам:

1. провера аутлајера
2. ако има аутлајера провера нормалности расподеле
3. ако нема нормалну онда стандардизација, ако има онда нормализација
4. додавање преосталих варијабли (које нисмо морали да стандардизујемо)
5. прављење тренинг и тест сета (createDataPartition u cheatsheet-u)
6. крос валидација (cross-validation u cheatsheet-u)
7. прављење предикције (knn u cheatsheet-u)
8. прављење конфузионе матрице (САМОСТАЛНО ПРАВИМО)
9. евалуација метрика (САМОСТАЛНО ПРАВИМО)
10. креирање новог модела са најбољим К параметром

```
#proveravamo da li neke variujable imaju autlajere

apply(X = data[,c(1,2,3,4,5,6)], 2, FUN = function(x) length(boxplot.stats(x)$out))
#vidimo da sve varijable imaju autlajere, potrebno je da ih standardizujemo
#proveravamo da li imaju normalnu raspodelu kako bismo znali na koji nacin cemo izvrsiti standardizaciju
apply(X = data[,c(1,2,3,4,5,6)], 2, FUN = function(x) shapiro.test(x))
#sve varijable imaju p < 0.05 sto znaci da nemaju normalnu raspodelu

data.std <- apply(X = data[,c(1,2,3,4,5,6)], 2, FUN = function(x) scale(x, center = median(x), scale = IQR(x)))
data.std <- as.data.frame(data.std)

#kreirali smo standardizovani dataframe
#za scale smo stavili IQR - interquartile range sto predstavlja razliku izmedju 3. i 1. kvartila i kao centar smo stavili
#ubacujemo u standardizovani dataset preostalu varijablu mode i izlaznu varijablu
data.std$mode <- data$mode
data.std$High_Streams <- data$High_Streams

#sa tako standardizovanim dataset-om kreiramo nas model
```

```
#kreiramo train i test setove
library(caret)
set.seed(1010) # allows for repeating the randomization process exactly
indexes <- createDataPartition(data.std$High_Streams, p = 0.8, list = FALSE)
train.data <- data.std[indexes, ]
test.data <- data.std[-indexes, ]

#radimo krosvalidaciju sa 10 iteracija
library(e1071)
library(caret)
numFolds = trainControl(method = "cv", number = 10)
kGrid =  expand.grid(.k = seq(from = 3, to = 25, by = 2))

#stavili smo neparne brojeve od 3 do 25 jer su nam oni potrebni kako bi jedna klasa bila dominantnija
knn.cv <- train(
  x = train.data[, -7],
  y = train.data$High_Streams,
  method = "knn", # use rpart() to build multiple
  trControl = numFolds, # <folds> from above
  tuneGrid = kGrid)
knn.cv
#dobili smo da nam je optimalna vrednost za parametar k = 7;
#mozemo je izuci direktno odatle
kValue <- knn.cv$bestTune$k
kValue
```

```r
library(class)
knn.pred <- knn(train = train.data[,-7], # training data without the output (class) variable
                test = test.data[,-7], # test data without the output (class) variable
                cl = train.data$High_Streams, # output (class) variable is specified here
                k = kValue)
#kreirali smo model knn i sada mozemo kreirati matricu konfuzije
knn.cm <- table(true = test.data$High_Streams, predicted = knn.pred)
knn.cm
```

НАПОМЕНА: код кнн нема модел одмах иде knn.pred, код cross валидације пазити након копирања из cheetsheet-а мора се мало модификовати (уместо cpGrid иде kGrid, уместо .cp иде .k, seq(ОВДЕ ИДУ НЕПАРНИ БРОЈЕВИ!), уместо class иде method="knn")

ТЕОРИЈА КОЈА ЈЕ БИТНА ЗА КОМЕНТАРИСАЊЕ РАДА:

Исто као за стабла одлучивања

Наивни Бајес

Тип података: улазне варијабле – факторске и нумеричке(ак нема нр. ДИСКРЕТИЗАЦИЈА)

излазна варијабла- факторска

Алгоритам:

1. провера нормалности нумеричих варијабли
2. ако нема нормалну онда дискретизујемо (discretize u cheatsheet-u)
3. додавање преосталих варијабли (које нисмо морали да дискретизујемо)
4. прављење тренинг и тест сета (createDataPartition u cheatsheet-u)
5. креирање модела (NaiveBayes u cheatsheet-u)
6. прављење предикције (prediction u cheatsheet-u) CLASS
7. прављење конфузионе матрице (САМОСТАЛНО ПРАВИМО)
8. евалуација метрика (САМОСТАЛНО ПРАВИМО)
9. прављење предикције (prediction u cheatsheet-u) RAW
10. roc крива (pROC u cheatsheet-u)
11. креирање новог модела са најбољим treshold параметром

```
# moramo prvo da izvrsimo proveru normalnosti raspodele za numericke varijable
num.vars <- c(2:7)
apply(data[,num.vars],MARGIN = 2,FUN = function(x) shapiro.test(x))
# Na osnovu shapiro testa zakljucujemo da:
# Varijable Size, Juiciness, Ripeness imaju normalnu raspodelu
# Varijable Weight, Sweetness, Crunchiness nemaju normalnu raspodelu
# Varijable koje imaju normalnu raspodelu mozemo da iskoristimo za kreiranje modela
# Varijable koje nemaju normalnu raspodelu moramo prvo da diskretizujemo
to_discretize <- c(3,4,5)
# pokusacemo da podelimo ukupan opseg vrednosti svake varijable na 5 intervala jednake ucestalosti
library(bnlearn)
discretized <- discretize(data[,to_discretize],
                          method = 'quantile',
                          breaks = c(5,5,5))
# vidimo da je diskretizacija uspesno izvrsena
summary(discretized)
data.new <- cbind(data[,c(1,2)], discretized, data[,c(6,7,8)])
```

```
# delimo dataset na trening (80% pocetnog dataseta) i test (20% pocetnog dataseta)
library(caret)
set.seed(10)
train_indices <- createDataPartition(data.new$IsGood, p = 0.8, list = FALSE)
train.data <- data.new[train_indices,]
test.data <- data.new[-train_indices,]
library(e1071)
# kreiramo model
nb1 <- naiveBayes(IsGood ~ ., data = train.data)
# vrsimo predvidjanje
nb1.pred <- predict(nb1, newdata = test.data, type = "class")
#kreiramo matricu konfuzije
nb1.cm <- table(true = test.data$IsGood, predicted = nb1.pred)
nb1.cm
```

НАПОМЕНА: кад прекопирамо за предикцију уместо raw стављамо class да бисмо видели предиктовану класу, а не број, е после тек кад одемо до трешхолда онда се користи raw, код roc response=**as.integer(izlazna varijabla)** , ако је код predictor= stoji [,1] онда се додаје labels=c(2,1) јер је позитивна класа у првој колони, кад стоји 2 онда је позитивна у другој колони и не мора ништа да се додаје.

```
# kreiramo predikcije u formi verovatnoca
nb2.pred.prob <- predict(nb1, newdata = test.data, type = "raw")
nb2.pred.prob
library(pROC)
# kreiramo roc krivu
nb2.roc <- roc(response = as.numeric(test.data$IsGood),
               predictor = nb2.pred.prob[,2])
nb2.roc$auc
# AUC je povrsina ispod ROC krive i ona nam govori koliko dobro mode
#razdvaja pozitivnu od negativne klase, kod nas iznosi 0.81
nb2.coords <- coords(nb2.roc,
                     ret = c("spec", "sens", "thr"),
                     x = "local maximas", transpose = FALSE)
nb2.coords
# koristimo youden metodu koja ce nam odrediti prag verovatnoce
#kojim se maksimizuje suma sensitivity i specificity metrika
plot.roc(nb2.roc,
         print.thres = TRUE,
         print.thres.best.method = "youden")
prob.threshold <- 0.451

# vrsimo ponovo predvidjanje sa novim pragom verovatnoce
nb2.pred2 <- ifelse(test = nb2.pred.prob[,2] >= prob.threshold,
                    yes = "Yes",
                    no = "No")
nb2.pred2 <- as.factor(nb2.pred2)
# kreiramo novu matricu konfuzije
nb2.cm2 <- table(actual = test.data$IsGood, predicted = nb2.pred2)
nb2.cm2
# ponovo racunamo evalucione metrike
nb2.eval2 <- compute.eval.metrics(nb2.cm2)

data.frame(rbind(nb1.eval, nb2.eval2), row.names = c("nb1", "nb2"))
```

ТЕОРИЈА КОЈА ЈЕ БИТНА ЗА КОМЕНТАРИСАЊЕ РАДА:

Исто као за стабла одлучивања + ово доле

```
# install.packages( pROC )
library(pROC)
nb2.roc <- roc(response = as.integer(test.set$High_Price),
               predictor = nb2.pred.prob[,2],
               levels = c(1,2))

# response je izlazna varijabla, ali vrednost treba da bude integer vrednost
# za predictor vrednost dajemo verovatnocu pozitivne klase, odnosno prva kolona
# levels je uredjen da prvo ide negativna, pa pozitivna klasa

# sensitivity odgovara recallu (TPR - true positive rate)
# u odnosu na sve automobile koji imaju visoku cenu, koji je udeo onih koje smo
# mi predvideli da imaju visoku cenu

# specificity je isto samo se odnosi na negativnu klasu (FPR - false positive rate)
# u odnosu na sve automobile koji imaju nisku cenu, koji je udeo onih koje smo
# mi predvideli da imaju nisku cenu

nb2.roc$auc
# sto je AUC - area under the curve veca, to se klasifikator smatra boljim,
# 0.9827 je u nasem slucaju (znaci da je izuzetno dobar model, sve preko 0.9 se smatraju bas dobrim modelima)
# sada pravimo plot da bismo videli threshold
```

Линеарна регресија

Тип података: улазне варијабле – нумеричке (факторске се бришу одмах)

излазна варијабла- нумеричка

Алгоритам:

1. провера корелисаности (cor u cheatsheet-u)
2. прављење тренинг и тест сета (createDataPartition u cheatsheet-u)
3. креирање модела (model u cheatsheet-u)
4. прављење предикције (prediction u cheatsheet-u)
5. прављење дијагностичких графикона (diagnostic u cheatsheet-u)
6. испитивање мултиколинеарности (multicolinearity u cheatsheet-u)
7. креирање новог модела са избаченим мултиколинеарним варијаблама

```
#Značajnost ostalih atributa se ispituje pomoću korelacione matrice
cor.mat <- cor(data.subset)
library(corrplot)
corrplot.mixed(cor.mat, tl.cex = 0.75, number.cex = 0.75)
#Na korelacionoj matrici se može primetiti da su za predviđaje atributa protein najznačajni(najveća korelisanost) atributi:
#   1)cholesterol - 0.87 (jedinično povećanje atributa cholesterol dovešće do povećanja atributa protein za 0.87)
#   2)calories - 0.8 (jedinično povećanje atributa calories dovešće do povećanja atributa protein za 0.8)
#   3)sodium - 0.76 (jedinično povećanje atributa sodium dovešće do povećanja atributa protein za 0.76)

#Zaključak: Za predviđanje atributa protein (prvi model LR) koristiće se atributi cholesterol,calories i sodium.
```

```
#KREIRANJE MODELA LR
#Podela dataseta na train i test
#Train set sadrzi 80% opservacija,a test ostalih 20%
library(caret)
set.seed(123)
i <- createDataPartition(data.subset$protein, p = 0.8, list = FALSE)
train <- data.subset[i, ]
test <- data.subset[-i, ]
#Kreiranje modela LR
lm1 <- lm(protein ~ cholesterol + calories + sodium, data = train)
summary(lm1)
```

```
lm1.pred <- predict(lm1, test)
lm1.pred
# DIJAGNOSTIČKI GRAFIKONI
par(mfrow = c(2,2))
plot(lm1)
par(mfrow = c(1,1))
```

```
#MULTIKOLINERANOST
library(car)
vif(lm1)
sqrt(vif(lm1))
#S obzirom da je za atribut calories sqrt(vif) vrednost >2,
#može se zaključiti da postoji multikorelisanost, koja može dovesti do nepouzdanih procena.
#Novi model za predikciju proteina treba kreirati sa atributima cholesterol i sodium.

lm2<-lm(protein ~ cholesterol+ sodium, data = train)
sqrt(vif(lm2))
#Multikolienearnost ne postoji više.
summary(lm2)
```

ТЕОРИЈА КОЈА ЈЕ БИТНА ЗА КОМЕНТАРИСАЊЕ РАДА:

comment("-**Резидуали** представљају разлику између стварних вредности зависне променљиве (protein) и предвиђених вредности које модел генерише. Вредност резидуала би требала да буде једнака нули.
-**Медијана резидуала** је -0.702. То значи да је половина наших података имала резидуалне вредности мање од -0.702, а друга половина веће.
-**Intercept** је 2.4256456, што значи да када су сви остали атрибути 0, очекујемо да је просечна вредност атрибута protein 2.4256456.
-Коефицијенти за променљиве cholesterol и sodium су статистички значајни јер су p-вредности далеко мање од 0.05, док коефицијент за calories није статистички значајан. Дакле cholesterol и sodium су релевантне за предикцију нивоа proteina у моделу.
-**Коефицијенти**: Када се остале променљиве држе константним:
Јединично повећање cholesterol за 1 довешће до повећања protein за 0.182 (ово значи да постоји позитивна линеарна веза између нивоа холестерола и протеина, тј. што је већи ниво холестерола, већи је и ниво протеина). Јединично повећање calories за 1 довешће до повећања protein за 0.0034, међутим с обзиром да calories није статистички значајан, не можемо да кажемо да ће значајно утицати на ниво протеина). Јединично повећање sodium за 1 довешће до повећања protein за 0.0083 (постоји позитивна линеарна веза између нивоа sodium-а и протеина).
-**Стандардна грешка резидуала** је 6.492, што нам даје информацију о просечној удаљености између стварних вредности и предвиђених вредности.
-**Коефицијент детерминације (R-квадрат)**: Модел објашњава око 83.52% варијабилитета protein променљиве.
-**F-статистика**: Нулта хипотеза F-статистике гласи да су сви коефицијенти независних променљивих нула, што значи да независне променљиве не утичу на зависну променљиву. С обзиром да је p-вредност мања од 0.05, може се закључити да модел има значајну предиктивну моћ и да га треба разматрати.")

comment("-**Reziduals vs Fitted** – приказује расподелу резидуала (разлике између стварних и предвиђених вредности) у односу на предвиђене вредности (да ли је задовољена претпоставка линеарности). Резидуали би требало да буду распоређени око хоризонталне линије (њихова вредност треба да буде 0). Са графикона може да се закључи да постоје мања одступања, тј. нелинеарност у подацима, што би могло довести до неадекватности модела.
-**Normal Q-Q** – говори о нормалној расподели резидуала. Тачке би требале да се прате дуж дијагонале. С обзиром да тачке одступају од дијагонале на крајевима, не можемо се у потпуности ослонити на модел.
-**Scale-Location** – приказује да ли варијабилитет (грешке, одступања) резидуала варирају на уједначен начин. Црвена линија треба да буде хоризонтална. На графикону варијанса резидуала није константна, што може довести до неадекватности модела.
-**Residuals vs. Leverage** – приказује екстремно високе и/или екстремно ниске вредности. Већина тачака требала би бити унутар граница, а екстремне вредности би требале бити ретке (није добро уколико постоје). На основу Кукова дистанце, можемо закључити да модел садржи outlayer.")

K means

Тип података: улазне варијабле – нумеричке (факторске се бришу одмах)

излазна варијабла- нумеричка

Алгоритам:

1. провера корелисаности (cor u cheatsheet-u)
2. провера да ли постоје аутлајери (apply sa boxplot.stats САМОСТАЛНО)
3. сређивање аутлајера (winsorize САМОСТАЛНО)
4. нормализација (utility.r је дато али научи за сваки случај)
5. elbow метода за најбољу вредност кластера
6. цртање графика
7. извршавање кластеризације
8. евалуација кластера

```
# sada cemo odraditi znacajnost varijabli kreiranjem matrice korelacije
library(ggplot2)
cm <- cor(df)
cm
library(corrplot)
corrplot.mixed(cm,  tl.cex = 0.7, number.cex = 0.7)
# vidimo da atributi kao sto su streams i in_apple_playlist su visoko korelisani sa sa nekim drugim atribu
# tako da cemo njih izbaciti iz daljeg razmatranja
df$streams <- NULL
df$in_apple_playlists <- NULL
# ostale atribute za sada zadrzavamo u daljoj analizi
# pre daljem rada, potrebno je proveriti postojanje autlajera (ekstremnih vrdnosti) u skupu podataka
# zbog nacina rada kmeans algoritma, autlajeri mogu znacajno uticati na dobijene klastere, te je njih potr
# obraditi na odgovoarajuci nacin
# proveravamo postojanje autlajera
apply(df, 2, FUN = function(x) length(boxplot.stats(x)$out))
# za kolone koje imaju ukupan broj autlajera veci od 10% od ukupnog broja obzervacija smo se odlucili
# da izbacimo iz daljeg razmatranja jer primenom tehnika za otklanjanje autlajera cemo znacajno uticati
# na raspodelu tih atributa te smatramo da takve kolone treba izbaciti iz daljeg razmatranja
df$released_year <- NULL
df$in_spotify_playlists <- NULL
df$speechiness <- NULL
# za ostale kolone, koje imaju manje od 10% autlajera, cemo primeniti tehniku Winsorize
library(DescTools)
# za svaki od preostalih atributa cemo iscrtati boxplot, videti sa koje strane se nalaze autlajeri
# i zameniti ih odgovarajucim vrednostima (najceste 5 i 95 percentilom u zavisnosti od vrednosti autlajera
boxplot(df$in_spotify_charts)
tmp <- Winsorize(df$in_spotify_charts, probs = c(0, 0.95))
boxplot(tmp)
# pomericemo jos nize granicu jer idalje postoje autlajeri
tmp <- Winsorize(df$in_spotify_charts, probs = c(0, 0.92))
boxplot(tmp)
```

```r
summary(df)
# sada cemo odraditi proces normalizacije kako bi sveli sve vrednosti na isti opseg
# a u cilju da zbog razlicitih velicina atributa neki atributi ne bi postali znacajniji samo
# zato sto imaju veci opseg
normalizuj <- function(x) {
  if(all(is.na(x) | x == 0)){
    return (x)
  }
  if (max(x) == min(x)) {
    return (x)
  }

  ( (x - min(x)) / (max(x) - min(x)) )
}

df2 <- as.data.frame(
  apply(df, 2, FUN = function(x) normalizuj(x))
)
summary(df2)
# sada ovako normalizovane vrednosti (vrednosti svih atributa svedeni na opseg od 0 do 1
# gde je 0 minimalna a 1 maksimalna vrednost) vracamo u pocetni skup podataka
df <- df2
summary(df)
# sada sprovodimo elbow metodu kako bi nasli najbolju vrednost za broj klastera
# prvo kreiramo pomocni dataframe u koje cemo smestati razlicite razultate za razlicite vrednosti klastera
df.tmp <- data.frame()
# odradicemo proveru za opseg od 2 do 10 klastera i videti koja vrednost od tih daje najbolje rezultate
for (k in 2:10) {
  model <- kmeans(x = df,
                  centers = k,
                  iter.max = 20,
                  nstart = 1000)
  df.tmp <- rbind(df.tmp,
                  c(k, model$tot.withinss, model$betweenss / model$totss))
}
df.tmp

# iscrtacemo grafik da zagledamo ponasanje modela za razlicite vrednosti parametra k
library(ggplot2)
ggplot(data = df.tmp, aes (x = k, y = tot.withinss)) + geom_line() + geom_point()
# sagledavajuci grafik kao i vrednosti koje se nalaze u df.tmp setu vidimo da su nam ponajbolje opcije za broj klastera 2 i
# s tim da za nijansu su bolje vrednosti za broj klastera = 2 te cemo tu vrednost uzeti za kreiranje model

# izvrsavamo klasterizaciju za broj klastera = 2
model <- kmeans(x = df,
                centers = 2,
                iter.max = 20,
                nstart = 1000)
model
# vidimo da sve obzervacije se nalaze ili u klasteru 1 ili u klasteru 2
```

ТЕОРИЈА КОЈА ЈЕ БИТНА ЗА КОМЕНТАРИСАЊЕ РАДА:

```r
source("Utility.R")
stats <- summary.stats(df, model$cluster, 2)
stats
# ucitana je skripta Utility i iz nje pozvana funkcija summary.stats koja nam daje odredjene statistike za model koji smo
# kreirali a na osnovu kojih cemo izvrsiti interpretaciju dobijenih klastera
# 1. Broj pesama po klasteru
# U klasteru 1 imamo 378 pesama, dok u klasteru 2 imamo 574, odnosno klaster 2 sadrzi vise pesama

# 2. Centri klastera
# U klasteru 1 se uglavnom nalaze pesme koje su visoko kotirane na applerang listi (in_apple_charts) i imaju veci bpm,
# odnosno brze su pesme. Sa druge strane, klaster 2 uglavnom sadrzi pesme nisko kotirane na applerang listi, koje
# imaju nesto sporiji ritam (bpm) i takodje nisu dobro kotirane na spotify rang listi

# 3. Disperzija od centara
# Za klaster 2 mozemo da kazemo da, iako je veci po svom obimu, je homogeniji jer je disperzija od centra klastera
# za skoro sve atribute manja u odnosu na model 1 a noricto atribute in_apple_charts i bpm koji su se pokazali kao
# znacajni za ovako kreiran model. Kod klastera 1, kojeg karakterise manja homogenost u pogledu na klaster 2,
# vidimo da za atribut in_spotify_charts imamo najvecu disperziju obzervacija u odnosu na centar klastera, sto bi znacilo
# da se u tom klasteru mogu naci i pesme koje imaju visoku poziciju na sporitfy rang listi ali i nisku poziciju


## Napomena: neke od prethodnih formulacija su blago izmenjene, bez promene znacenja, kako bi se ucinile jasnijim i/ili
## kompletnijim. VAZNO: date interpretacije tj nacin na koji su iskazane nije jedini korektan nacin, vec pokazuje samo
## jedan od nacina da se formulisu interpretacije metrika kako bi se odgovorilo na zahtev zadatka.
##
```